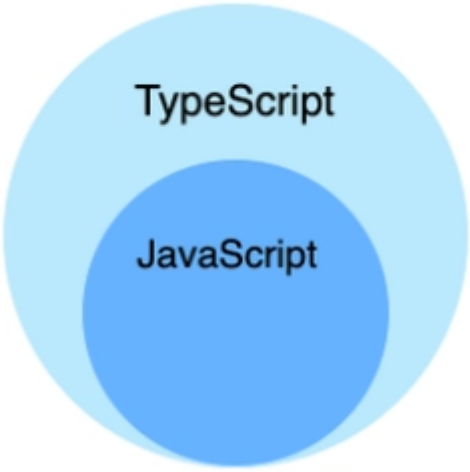
- 引言
  - 一些大厂已经把ts作为默认开发语言，很多中大型项目，包括我们用编译器、技术栈、组件库、插件等，也都是用ts编写的，ts势在必学。
  - 为什么js用得好好的，突然出现了ts?
    - js类型灵活，但随着项目规模的增大，这种灵活反而成为了负担
    - 很多类型错误写的时候发现不了，运行的时候才会出错
- ts介绍
  - ts定义
    - ts是js的超集，支持所有js语法，但在js语法的基础上引入了类型相关的新语法。



  - ts优势
    - 丰富的语法提示，提高开发效率

```
// 变量
const user: {
  name: string
  age: number
} = {
  name: 'xiaoming',
  age: 20
}
user.age.toFixed(0)

export {}
```

    - 编译时类型校验，减少运行时bug的产生

```
// 函数
```

```
function getPx(param: number) {
  return param + 'px'
}
const width: string = getPx(1)

export {}
```

- ts学习过程
  - 学习基础语法和内置工具类型
    - 基本类型

```
// 原始数据类型，都要采用小写类型
let name: string = 'Sun'
let age: number = 20
let male: boolean = true
let something: any = 123
```

    - 数组

```
const arr1: number[] = [1, 2, 3]
const arr2: string[] = ['a', 'b', 'c']
const arr3: (number | string)[] = [1, 2, 3, 'a', 'b', 'c'] // “|”是联合类型语法，表示这几种类型都可以
// 或
const arr4: Array<number> = [1, 2, 3] // 采用泛型来声明数组，其中“<>”是泛型语法
```

    - 类型别名

```
type Direction = 'top' | 'right' | 'bottom' | 'left'
let direction: Direction = 'top'
```

    - 对象类型
      - 常规属性声明

```
type User = {
```

```
  name: string
  age: number
}

const user: User = {
  name: 'Sun',
  age: 20
}
```

- 可选属性声明

```
type User = {
  name: string
  age: number
  motorbike?: string
}

const user1: User = {
  name: 'Sun',
  age: 20
}
const user2: User = {
  name: 'Sun',
  age: 20,
  motorbike: 'haojue'
}
```

- 任意属性声明

```
type User = {
  name: string
  age: number
  [key: string]: any
}

const user: User = {
  name: 'Sun',
  age: 20,
```

```
  qq: 790964731,
  weixin: 'sjf_08'
}
```

        - 键可以写成任意内容，但习惯用key
        - 键的类型可以是string、number、symbol
        - 值的类型必须包含已有属性的类型
    - 函数类型
        - 写在函数声明里

```
function sum(a: number, b:number): number {
  return a + b
}
sum(1, 2)
```

        - 使用类型别名

```
type Sum = (a: number, b:number) => number
const sum2: Sum = (a, b) => a + b
sum2(1, 2)
```

        - 可选参数

```
// 如果有其他必选参数，必选参数必须放在可选参数之前
function sum(a: number, b?:number): number {
  return a + (b ?? 0)
}
sum(1, 2)
sum(1)
```

    - 文档
        - TypeScript
        - TypeScript中文网
        - TypeScript（冴羽）
- 了解项目所用技术栈、组件库提供的工具类型
    - 文档

        - TypeScript 与组合式 API
- ts应用场景
    - 原则：谁用数据，谁定义类型。别人要想传数据过来，必须按照用数据的地方定义的类型来传
    - 举例：
        - 接口要参数，定义类型，用这个接口就得按这个类型传参
        - 组件要用数据，定义类型，用这个组件就得按这个类型传参
        - 页面展示要用数据，定义类型，接口返回后的数据，按定义的类型处理好后，再给响应式变量赋值
- 结语
    - ts将js的“灵活性”限制在了规则之内，让一切变得规范、可控。尤其在大型项目中，ts的优势更为明显。